

Post-Mortem Analysis and Simulation: The Virtues and Limitations of ACID in Hyper-Scale Systems

Zachary Frappier

September 6, 2026

Abstract

The 2021 Robinhood/Gamestop trading halt was a controversial incident that resulted in protecting institutional hedge funds from losing billions at the cost of retail investor's money and trust in what they would deem a rigged system. I will be exploring how the system entered this invalid state that led to the shut down involving high-concurrency transactions, clearing obligations, and platform availability. Using Gray's ACID transaction theory and Chen's entity-relationship modeling framework, this paper develops a conceptual brokerage ERD and a small-scale concurrency simulation [1, 4]. The results show that unenforced isolation produces invalid states such as negative cash balances, inflated positions, and over committed clearing capacity, while strict isolation preserves correctness at the cost of increased latency and reduced throughput. The paper concludes that hyper-scale financial systems require transaction boundaries, reservation holds, idempotent order processing, risk throttling, and reconciliation mechanisms to balance consistency, availability, and user trust.

1 Introduction

Hedge funds profit off of businesses they think will fail by short selling, a process where they borrow a share, and buy it back later if the price falls. A community of reddit retail investors decided to compete against the hedge funds by buying GameStop stock in masses, driving the price up. The hedge fund, Melvin Capital, received a \$2.75 billion private investment from Citadel and Point72 to bail them out. Melvin Capital is still locked into the stock, conveniently while facing this severe loss, Robinhood's app 'crashes'. But only the buy option for the GameStop stock is affected. No one can buy more stock so the value goes down. Robinhood attributes this restriction to being unable to post sufficient collateral at clearing houses to execute client orders, that is, Robinhood was unable to pay a middleman, clearinghouse, to protect its number of transactions [2]. They were forced to shut off the buy option. From a database systems perspective,

a GME buy order is not a single isolated action. It affects customer cash, order status, trade execution records, positions, and clearing obligations. Gray’s transaction theory explains why atomicity, consistency, isolation, and durability are necessary to preserve valid financial state [1]. However, the incident also illustrates Gray’s broader warning that transactions become difficult when systems are distributed, high-volume, and tied to real-world obligations [1, 3]. This paper argues that ACID guarantees are necessary for financial correctness, but strict isolation under extreme concurrency can create bottlenecks that force business-level restrictions.

2 Background

The ERD models the brokerage platform as a set of connected business entities [4]. A User owns one or more BrokerageAccounts, and each BrokerageAccount may place many Orders. Each Order targets one Security, such as GME, and may produce one or more TradeExecutions depending on whether the order is filled fully or partially. Executed trades update a customer Position, while the account records related CashLedger entries. Finally, each TradeExecution may create downstream ClearingObligations, which represent the settlement and collateral pressure central to the GameStop incident.

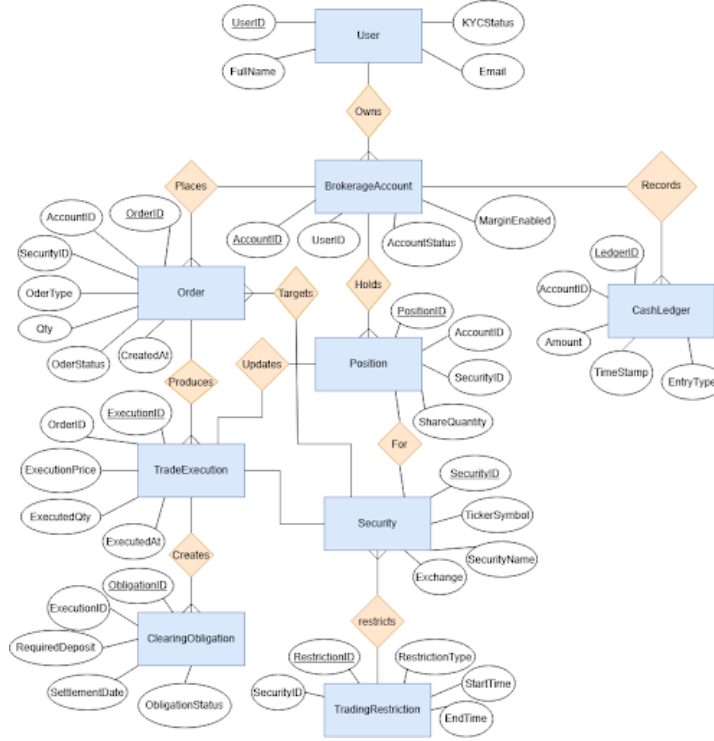


Figure 1: Conceptual entity-relationship diagram for the simulated brokerage trading system.

3 Methodology

The full Python simulation script and generated result files are available in the project repository.¹ The simulation was implemented using in-memory objects that correspond to the ERD entities: BrokerageAccount, Security, Order, Position, CashLedgerEntry, and ClearingObligation [4]. Each thread represented a concurrent request to buy one share of GME. The transaction followed a three-step read-delay-write structure. First, the system checked whether GME was tradable, whether the account had sufficient cash, and whether clearing capacity remained available. Second, the system introduced a synthetic delay to simulate server processing, network latency, and database response time. Third, the system attempted to create an order, update the position, record a cash ledger entry, and create a clearing obligation.

Phase 1 executed the transaction without isolation, allowing all threads to read and write shared state concurrently. Phase 2 used a mutex lock around the full transaction block to simulate strict isolation. Both phases were tested

¹<https://github.com/zacfrappier/Robinhood-Gamestop-ACID-Concurrency-Simulation>

at 10, 100, and 1,000 concurrent threads. The independent variable was the number of concurrent threads, while the dependent variables were data accuracy, throughput, and average latency.

4 Results

```

C:\Users\chris\Documents\Projects\Blockchain-Simulators\4038-concurrency-simulation_python\simulation.py
threads | phase | accepted_orders | rejected_orders | integrity_violations | throughput_tx_per_sec | avg_latency_ms | max_latency_ms | final_cash | final_gme_shares | clearing_capacity_remaining
10 | no_isolation | 38 | 0 | 0 | 1000.00 | 35.46 | 35.46 | 17000.0 | 1.0 | 3700.0
10 | strict_isolation | 38 | 0 | 0 | 99.04 | 86.87 | 86.87 | 100.0 | 1.0 | 3700.0
100 | no_isolation | 380 | 0 | 0 | 4300.74 | 38.43 | 38.43 | 13100.0 | 1.0 | 3700.0
100 | strict_isolation | 380 | 0 | 0 | 98.04 | 838.14 | 838.14 | 618.47 | 1.0 | 3700.0
1000 | no_isolation | 4300 | 0 | 0 | 1100.0 | 4.46 | 46.78 | 20000.0 | 1.0 | 3700.0
1000 | strict_isolation | 380 | 0 | 0 | 98.17 | 362.71 | 408.05 | 6.0 | 1.0 | 3700.0

```

Figure 2: Terminal output from the concurrency simulation showing accepted orders, rejected orders, integrity violations, throughput, latency, final cash balance, final GME share count, and remaining clearing capacity across no-isolation and strict-isolation phases.

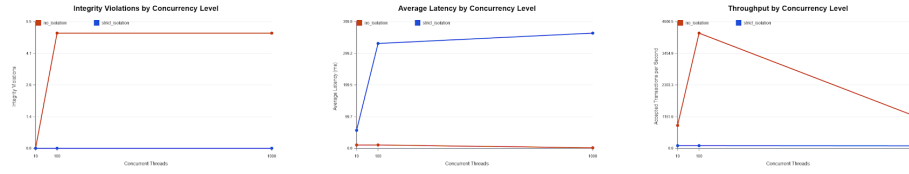


Figure 3: Comparison of no-isolation and strict-isolation transaction behavior across increasing concurrency levels. Strict isolation prevents integrity violations but increases latency and reduces throughput.

The integrity-violation graph shows that strict isolation eliminated invalid final states, while the no-isolation phase produced violations under higher concurrency. The latency graph shows the cost of this correctness: strict isolation increased average latency because requests waited for access to the transaction block. The throughput graph shows the same trade-off from another angle. The no-isolation phase completed more accepted transactions per second, but that throughput was misleading because many accepted transactions violated financial constraints.

5 Discussion

The experiment supports Gray’s argument that transaction systems are necessary because they prevent partial or conflicting updates from leaving the database in an invalid state [1]. In the no-isolation phase, the system behaved as if speed were the only goal, but the result was a corrupted final state. In the strict-isolation phase, the system preserved correctness, but performance

declined because requests were serialized. This a challenge for hyper-scale financial systems: absolute consistency protects ledgers and customer assets, but strict coordination can reduce availability during extreme demand.

The DynamoDB study shows that modern systems do not have to choose between no isolation and a single global lock [3]. Instead, scalable systems can use transaction coordinators, timestamp ordering, conditional writes, idempotent requests, and conflict rejection to preserve ACID properties while reducing bottlenecks. For a brokerage platform, these ideas support architectural patterns such as cash reservation holds, clearing-capacity checks before order acceptance, queued order states, and reconciliation ledgers.

6 Conclusion

The Robinhood/GME incident demonstrates the tension between financial integrity, platform availability, and public trust. The simulation shows that removing isolation can increase apparent throughput, but only by allowing invalid states such as negative cash balances, inflated positions, and overcommitted clearing capacity. Strict isolation preserves correctness but increases latency and reduces throughput under high concurrency. Gray’s ACID theory remains essential for preserving valid financial state, but modern hyper-scale systems require more refined architectures than a single lock or unrestricted concurrency [1, 3]. Reservation holds, idempotent order processing, risk-aware throttling, transaction ledgers, and reconciliation systems provide a more realistic path for preserving consistency while maintaining availability during market stress. The incident also shows that technical controls must be paired with transparent communication, because users are unlikely to trust a platform when restrictions appear to benefit institutional participants at the expense of retail customers.

7 References

References

- [1] J. Gray, “The transaction concept: Virtues and limitations,” in *Proc. 7th Int. Conf. Very Large Data Bases (VLDB)*, Cannes, France, 1981, pp. 144–154. [Online]. Available: <https://www.cs.utexas.edu/~witchel/S25-380L/papers/gray81vldb-transaction.pdf>
- [2] R. Garvey, J. He, and F. Wu, “Retail broker trading restrictions and market liquidity: An examination of GameStop,” *Applied Economics*, vol. 56, no. 34, pp. 4140–4153, 2024, doi: 10.1080/00036846.2023.2210819.
- [3] J. Idziorek, A. Keyes, C. Lazier, S. Perianayagam, P. Ramanathan, J. C. Sorenson III, D. Terry, and A. Vig, “Distributed transactions at scale in Amazon DynamoDB,” in *Proc. 2023 USENIX Annual Technical Conf.*

(*USENIX ATC 23*), Boston, MA, USA, 2023, pp. 705–717. [Online]. Available: <https://www.usenix.org/system/files/atc23-idziorek.pdf>

- [4] P. P.-S. Chen, “The entity-relationship model: Toward a unified view of data,” *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, Mar. 1976, doi: 10.1145/320434.320440.